Инструкция для системного администратора автоматизированной системы генерации персонализированных рецептов спортивного питания (АС ГРСН)

(составил студент Привалихин Дмитрий, ИСИб-23-1)

Содержание

1 Общие сведения 3
1.1 Назначение инструкции 3
1.2 Состав системы 3
1.3 Требования к серверу 3
2 Установка системы 5
2.1 Подготовка сервера 5
2.2 Получение дистрибутива 5
2.3 Автоматическая установка 5
2.4 Ручная установка 6
3 Настройка системы 7
3.1 Файл настроек 7
3.2 Подключение GigaChat 8
3.3 Настройка HTTPS 8
3.4 Резервные копии на отдельном диске 9
4 Проверка работоспособности 9
5 Резервное копирование и восстановление 9
5.1 Автоматическое копирование 9
5.2 Создание копии вручную 10
5.3 Восстановление базы данных 10
6 Работа с базой данных 11
7 Модуль администрирования 12
8 Обслуживание системы 13
9 Типовые неисправности 16

1 Общие сведения

1.1 Назначение инструкции

Инструкция предназначена для системного администратора организации, которая приобрела автоматизированную систему генерации персонализированных рецептов спортивного питания (АС ГРСН). В инструкции описаны подготовка сервера, установка и первый запуск системы, настройка параметров, проверка работоспособности, резервное копирование и восстановление базы данных, обслуживание и устранение типовых неисправностей.

Работа с данными внутри системы (пользователи, справочники, журнал событий) выполняется в веб-интерфейсе пользователем с ролью «администратор»; этот модуль кратко описан в разделе 7. Работа обычных пользователей описана в отдельной инструкции для пользователей.

1.2 Состав системы

Система поставляется в виде исходного кода с файлами сборки и разворачивается в контейнерах Docker. Состав контейнеров приведён в таблице 1.1, схема взаимодействия – на рисунке 1.1.

Таблица 1.1 – Состав контейнеров АС ГРСН

Контейнер	**Образ, технологии**	**Назначение**
db	PostgreSQL 15 (Alpine Linux)	База данных системы. Данные хранятся в томе Docker pgdata
backend	Python 3.12, FastAPI, Uvicorn (Debian 12)	REST API: бизнес-логика, расчёты, обращение к модели GigaChat, формирование отчётов, копирование и восстановление базы. При запуске применяет миграции и заполняет справочники
frontend	Nginx 1.27 (Alpine Linux)	Веб-интерфейс и обратный прокси: отдаёт страницы и передаёт запросы /api в backend; работает по HTTP или HTTPS
backup	PostgreSQL 15 (Alpine Linux), cron	Ежедневное резервное копирование базы утилитой pg_dump и удаление старых копий

Внешняя система – GigaChat API (ПАО «Сбербанк»): модель GigaChat-2-Pro формирует варианты рецептов. Если ключ доступа к GigaChat не задан или сервис недоступен, рецепты формирует встроенный резервный генератор, и система продолжает работать.

1.3 Требования к серверу

Требования к серверу соответствуют техническому заданию:

– процессор – от 2 ядер;
– оперативная память – от 4 ГБ;
– дисковое пространство – от 20 ГБ на системном диске;
– отдельный диск (или отдельный раздел другого физического диска) для резервных копий;

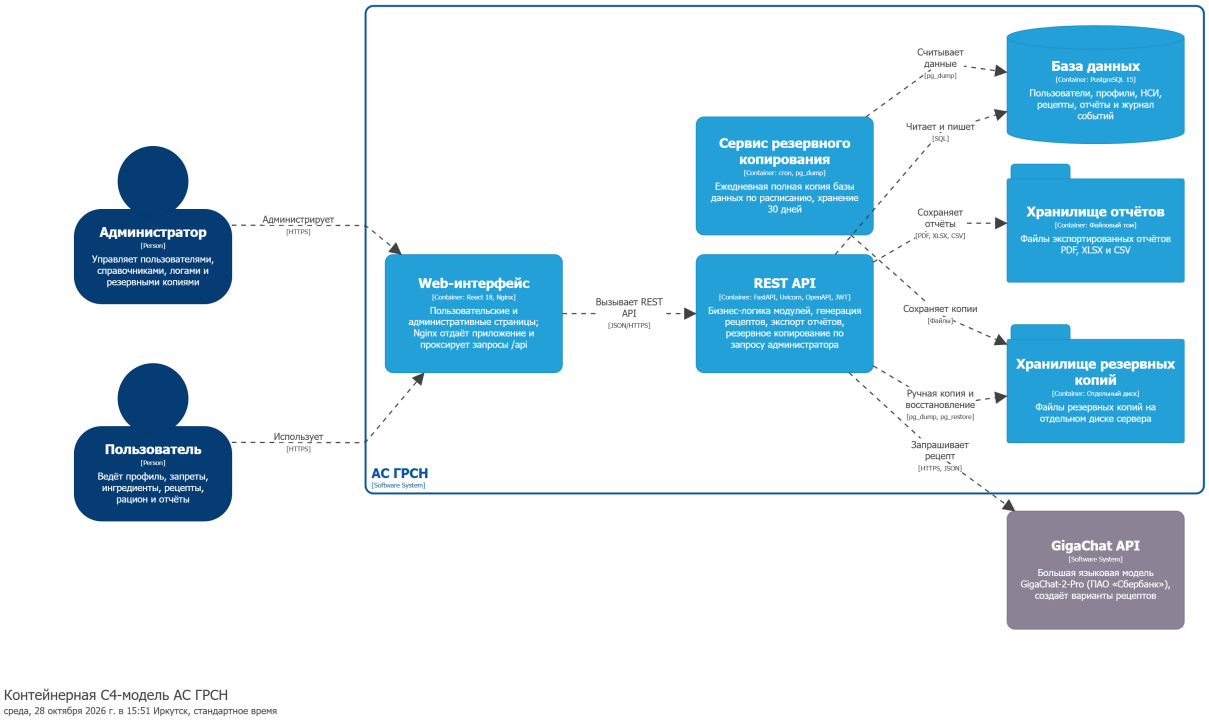


Рисунок 1.1 – Схема контейнеров АС ГРСН

– операционная система, на которой работает Docker: Linux-сервер с Docker Engine и плагином Docker Compose либо Windows 10/11 или macOS с Docker Desktop;
– доступ в интернет для загрузки образов при установке и для обращения к GigaChat API (адреса `ngw.devices.sberbank.ru`, порт 9443, и `gigachat.devices.sberbank.ru`, порт 443);
– свободные сетевые порты: для веб-интерфейса (по умолчанию 8080 – HTTP и 8443 – HTTPS) и порт базы данных для локального подключения администратора (по умолчанию 5432, доступен только с самого сервера).

Ресурсы между контейнерами распределены так, чтобы система укладывалась в минимальную конфигурацию сервера (таблица 1.2). Значения задаются в файле настроек `.env` (раздел 3).

Таблица 1.2 – Ресурсы контейнеров

Контейнер	**Процессор, ядер**	**Память**	**Параметры в .env**
db	0,75	1,5 ГБ	DB_CPUS, DB_MEMORY
backend	1	1,75 ГБ	BACKEND_CPUS, BACKEND_MEMORY
frontend	0,15	256 МБ	FRONTEND_CPUS, FRONTEND_MEMORY
backup	0,1	512 МБ	BACKUP_CPUS, BACKUP_MEMORY
Итого	**2**	**4 ГБ**	

2 Установка системы

2.1 Подготовка сервера

1) Установите Docker:
   - на Linux-сервере – Docker Engine и плагин Docker Compose по официальной инструкции https://docs.docker.com/engine/install/;
   - на Windows или macOS – Docker Desktop (https://www.docker.com/products/docker-desktop/). После установки запустите Docker Desktop и дождитесь состояния «Engine running».
2) Установите Git (https://git-scm.com/).
3) Подключите и подготовьте отдельный диск для резервных копий, создайте на нём папку, например `/mnt/backup/grsn` на Linux или `D:\grsn_backups` на Windows. На Linux выдайте права на запись пользователю с идентификатором 1001, под которым работает API:

```
sudo mkdir -p /mnt/backup/grsn
sudo chown 1001:1001 /mnt/backup/grsn
```

4) Проверьте, что Docker работает:

```
docker info
docker compose version
```

2.2 Получение дистрибутива

Загрузите дистрибутив системы и перейдите в его каталог:

```
git clone --branch 2.0mb_norm --single-branch \
    https://github.com/Skipl1/ASOIU.git
cd ASOIU/Optimization-Methods/grsn_app
```

Все дальнейшие команды `docker compose` выполняются в этом каталоге (`grsn_app`).

2.3 Автоматическая установка

Для установки одной командой запустите скрипт установки:

```
# Linux, macOS, Git Bash
bash scripts/setup.sh

# Windows (PowerShell)
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
```

Скрипт выполняет следующие действия:

1) проверяет, что Docker установлен и запущен (на Windows при необходимости запускает Docker Desktop);
2) создаёт файл настроек `.env` из шаблона `.env.example`: генерирует случайные пароль базы данных, пароль учётной записи приложения и секрет подписи токенов (`JWT_SECRET`), подбирает свободные порты;
3) собирает и запускает контейнеры (`docker compose up -d --build`); первая сборка занимает 5–15 минут;
4) ждёт готовности системы (до 5 минут) и проверяет вход;
5) выводит адрес сайта, учётные записи и основные команды управления.

Параметры скрипта приведены в таблице 2.1.

Таблица 2.1 – Параметры скрипта установки

Linux / macOS	**Windows**	**Назначение**
--gigachat-key "<ключ>"	-GigaChatKey "<ключ>"	Записать ключ GigaChat в .env
--https	-Https	Включить HTTPS
--no-browser	-NoBrowser	Не открывать браузер после установки
--reset	-Reset	Удалить контейнеры и **все данные** и установить заново

Повторный запуск скрипта безопасен: существующий файл `.env` и данные не изменяются.

Важно. Учётная запись администратора системы создаётся один раз – при первом запуске – из параметров `ADMIN_EMAIL`, `ADMIN_PASSWORD` и `ADMIN_NAME`. Перед первым запуском в рабочей среде задайте в `.env` собственный адрес и надёжный пароль администратора и отключите демонстрационные данные (`SEED_DEMO=false`), см. раздел 3. При каждом запуске контейнер `backend` проверяет, есть ли пользователь с адресом `ADMIN_EMAIL`, и создаёт его, только если такого нет. Поэтому изменение `ADMIN_PASSWORD` после первого запуска пароль существующего администратора не меняет, а указание нового адреса `ADMIN_EMAIL` приводит к созданию ещё одной учётной записи администратора.

2.4 Ручная установка

Тот же результат без скрипта:

1) Создайте файл настроек из шаблона:

```
cp .env.example .env
```

2) Откройте `.env` в текстовом редакторе и задайте как минимум `POSTGRES_PASSWORD`, `JWT_SECRET`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` и `BACKUP_PATH` (раздел 3). Для генерации секрета можно использовать команду:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

3) Соберите и запустите контейнеры:

```
docker compose up -d --build
```

4) Откройте в браузере адрес `http://<адрес сервера>:8080` (или `https://<адрес сервера>:8443` при включённом HTTPS) и войдите под учётной записью администратора.

При запуске контейнер `backend` автоматически применяет миграции базы данных, создаёт ограниченную учётную запись приложения в PostgreSQL и заполняет справочники (фитнес-ориентиры, категории, аллергены, ингредиенты).

3 Настройка системы

3.1 Файл настроек

Все параметры системы задаются в файле `.env` в каталоге `grsn_app`. Файл содержит пароли и ключи, поэтому его нельзя передавать посторонним и добавлять в систему контроля версий. Основные параметры приведены в таблице 3.1.

Таблица 3.1 – Основные параметры файла `.env`

Параметр	**Назначение**
База данных	
POSTGRES_USER, POSTGRES_DB	Владелец и имя базы данных PostgreSQL
POSTGRES_PASSWORD	Пароль владельца базы данных (обязательный)
APP_DB_PASSWORD	Пароль ограниченной учётной записи приложения grsn_app; если не задан, вычисляется из POSTGRES_PASSWORD
Безопасность	
JWT_SECRET	Секрет подписи токенов доступа – длинная случайная строка
ACCESS_TOKEN_MINUTES	Время жизни сессии без активности, минут (по умолчанию 30)
MAX_FAILED_LOGINS, LOCKOUT_MINUTES	Число неудачных попыток входа и время блокировки, минут (5 и 15)
Модель генерации	
LLM_PROVIDER	gigachat – GigaChat API; mock – только встроенный генератор
GIGACHAT_CREDENTIALS	Ключ авторизации GigaChat API
GIGACHAT_MODEL	Модель: GigaChat-2-Pro (основная) или GigaChat-2
LLM_MAX_CONCURRENCY	Число одновременных запросов к модели (тариф Freemium – 1)
LLM_FALLBACK_TO_MOCK	Использовать резервный генератор при недоступности модели (true)
Начальное наполнение	
ADMIN_EMAIL, ADMIN_PASSWORD, ADMIN_NAME	Учётная запись администратора, создаваемая при первом запуске
SEED_DEMO	Создавать демонстрационного пользователя (true/false); в рабочей среде – false
Веб-сервер и сеть	
ENABLE_HTTPS	true – работа по HTTPS, HTTP перенаправляется на HTTPS
HTTP_PORT, HTTPS_PORT	Порты веб-интерфейса на сервере (8080 и 8443)
SERVER_NAME	Имя сервера (домен) для сертификата

Параметр	Назначение
DB_PORT	Порт PostgreSQL для локального подключения администратора (5432)
Резервное копирование и прочее	
BACKUP_PATH	Папка резервных копий на отдельном диске
BACKUP_CRON	Расписание копирования в формате cron (по умолчанию 0 3 * * * – ежедневно в 03:00)
BACKUP_RETENTION_DAYS	Срок хранения копий, дней (30)
APP_TIMEZONE	Часовой пояс (Europe/Moscow)
API_WORKERS	Число процессов API (2)
*_CPUS, *_MEMORY	Ресурсы контейнеров (таблица 1.2)

После изменения `.env` примените настройки командой:

```
docker compose up -d
```

Docker пересоздаст только те контейнеры, параметры которых изменились; данные при этом сохраняются.

3.2 Подключение GigaChat

1) Зарегистрируйтесь на портале https://developers.sber.ru и создайте проект GigaChat API.
2) В настройках проекта получите ключ авторизации (Authorization key).
3) Запишите ключ в `.env`: `GIGACHAT_CREDENTIALS=<ключ>`. Для тарифа для юридических лиц укажите соответствующее значение `GIGACHAT_SCOPE` и при необходимости увеличьте `LLM_MAX_CONCURRENCY`.
4) Перезапустите API:

```
docker compose up -d backend
```

Корневой сертификат Минцифры, необходимый для соединения с GigaChat, входит в поставку. Проверить подключение можно по ответу служебного адреса проверки (раздел 4): поле `llmProvider` должно иметь значение `gigachat`.

3.3 Настройка HTTPS

1) Задайте в `.env` параметры `ENABLE_HTTPS=true` и `SERVER_NAME=<домен сервера>`.
2) Поместите сертификат и закрытый ключ, выданные для домена, в каталог `frontend/nginx/certs` под именами `server.crt` и `server.key`.
3) Перезапустите веб-сервер:

```
docker compose up -d frontend
```

Если файлы сертификата не найдены, при запуске создаётся самоподписанный сертификат: соединение будет зашифровано, но браузер покажет предупреждение. Такой режим допустим только для проверки. После включения HTTPS сайт доступен по адресу `https://<домен>:8443`, а

обращения по HTTP перенаправляются на HTTPS. Для работы на стандартных портах задайте `HTTP_PORT=80` и `HTTPS_PORT=443`.

3.4 Резервные копии на отдельном диске

Задайте в `.env` путь к папке на отдельном диске, подготовленной в разделе 2.1, например `BACKUP_PATH=/mnt/backup/grsn` (Linux) или `BACKUP_PATH=D:/grsn_backups` (Windows), и выполните `docker compose up -d`. Если параметр не задан, копии хранятся в томе Docker на том же диске, что и база данных; такой режим не соответствует требованиям к надёжности.

4 Проверка работоспособности

1) Проверьте состояние контейнеров:

```
docker compose ps
```

Все четыре контейнера должны быть в состоянии `Up`, контейнеры `db` и `backend` – с отметкой `(healthy)`:

```
NAME              SERVICE    STATUS
grsn-backend-1    backend    Up (healthy)
grsn-backup-1     backup     Up
grsn-db-1         db         Up (healthy)
grsn-frontend-1   frontend   Up
```

2) Проверьте служебный адрес проверки доступности API и базы данных:

```
curl http://localhost:8080/api/v1/health
{"status":"ok","llmProvider":"gigachat","llmModel":"GigaChat-2-
Pro"}
```

3) Проверьте ресурсы контейнеров:

```
bash scripts/show_limits.sh
# Windows:
powershell -ExecutionPolicy Bypass -File scripts\show_limits.ps1
```

Скрипт выводит лимиты процессора и памяти каждого контейнера, текущее потребление и итог (2 ядра, 4 ГБ). Нагрузку в реальном времени показывает команда `docker stats`.

4) Откройте сайт в браузере, войдите под учётной записью администратора и убедитесь, что открывается раздел «Администрирование» (раздел 7). Документация API доступна по адресу `/api/docs`.

5 Резервное копирование и восстановление

5.1 Автоматическое копирование

Контейнер `backup` по расписанию `BACKUP_CRON` (по умолчанию ежедневно в 03:00) создаёт полную копию базы данных утилитой `pg_dump` в формате PostgreSQL custom. Файлы называются `grsn_<дата>_<время>_auto.dump` и сохраняются в папку `BACKUP_PATH`. Копии старше `BACKUP_RETENTION_DAYS` дней удаляются автоматически. Результат каждого запуска записывается в журнал контейнера:

```
docker compose logs backup
```

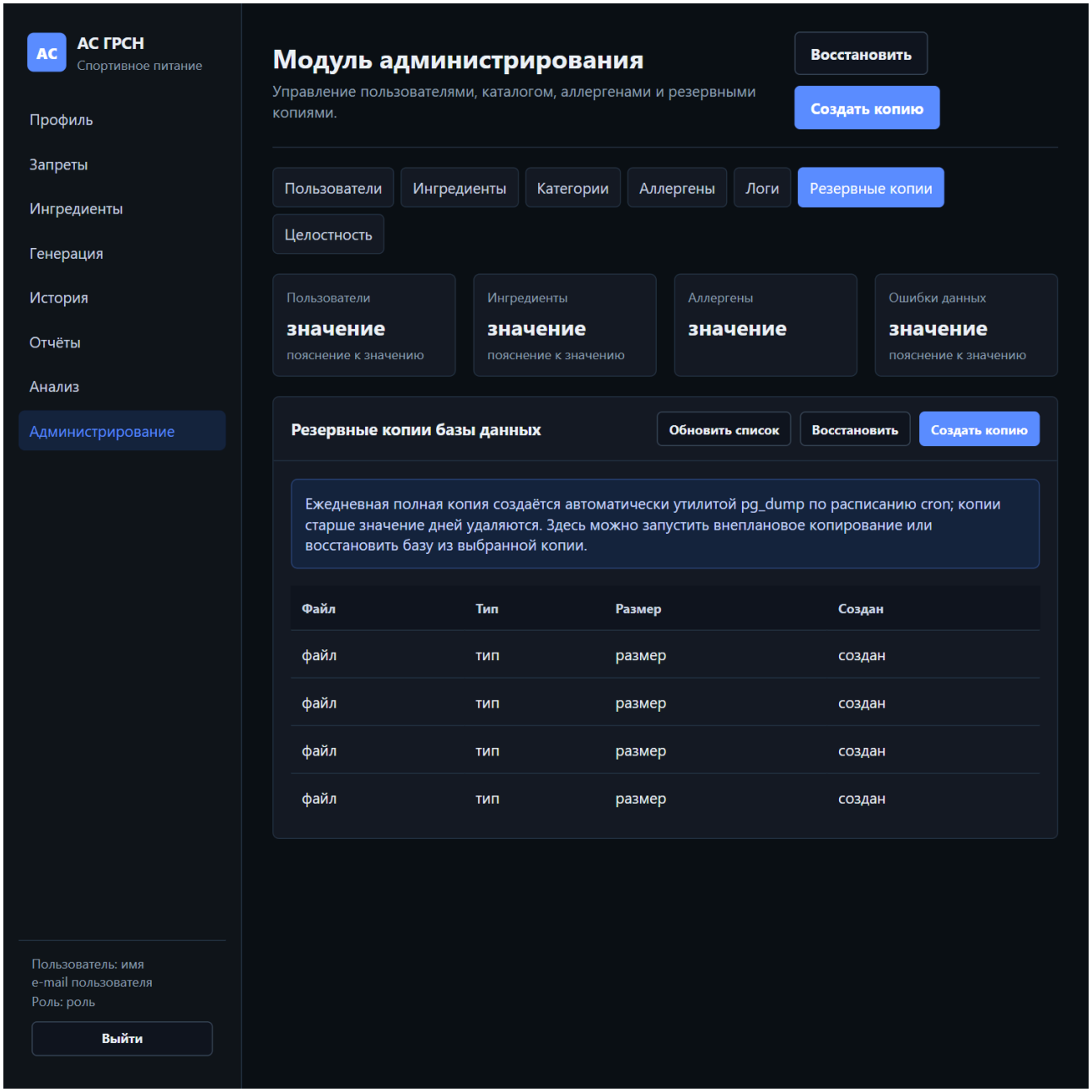


Рисунок 5.1 – Вкладка «Резервные копии»

5.2 Создание копии вручную

Внеплановую копию можно создать двумя способами:

– в веб-интерфейсе: «Администрирование» → вкладка «Резервные копии» → кнопка «Создать копию» (рисунок 5.1). Копия получит имя `grsn_<дата>_<время>_manual.dump`, событие запишется в журнал событий системы;

– из консоли сервера:

```
docker compose exec backup sh /scripts/backup.sh
```

5.3 Восстановление базы данных

Восстановление заменяет все данные системы состоянием на момент создания копии. Перед восстановлением рекомендуется создать свежую копию текущего состояния.

Восстановление из веб-интерфейса:

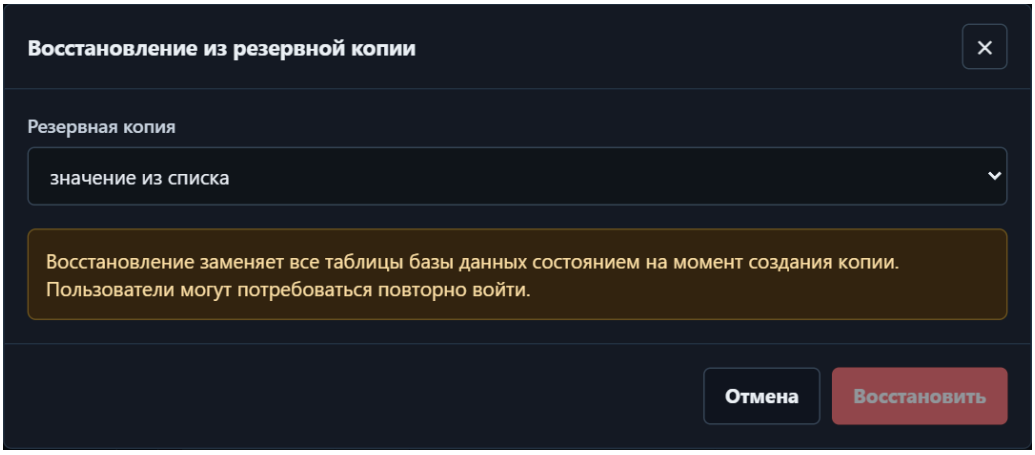


Рисунок 5.2 – Окно «Восстановление из резервной копии»

1) Откройте «Администрирование» и нажмите кнопку «Восстановить».
2) В окне «Восстановление из резервной копии» (рисунок 5.2) выберите копию в списке «Резервная копия».
3) Нажмите кнопку «Восстановить» и дождитесь сообщения о результате. Пользователям может потребоваться войти в систему заново.

Восстановление из консоли сервера (например, если веб-интерфейс недоступен):

```
docker compose exec backup ls /backups
docker compose exec backup pg_restore -h db -U grsn -d grsn \
    --clean --if-exists --no-owner --single-transaction /backups/
<имя файла>
docker compose restart backend
```

Для переноса системы на другой сервер установите систему на новом сервере (раздел 2), скопируйте файл копии в папку `BACKUP_PATH` нового сервера и выполните восстановление.

6 Работа с базой данных

Для просмотра данных и выполнения SQL-запросов можно использовать pgAdmin, DBeaver или другой клиент PostgreSQL. Порт базы данных открыт только на самом сервере (`127.0.0.1`), поэтому клиент запускается на сервере либо подключается через SSH-туннель. Параметры подключения:

– хост – `127.0.0.1`, порт – значение `DB_PORT` (по умолчанию 5432);
– база данных – `POSTGRES_DB` (`grsn`);
– пользователь и пароль – `POSTGRES_USER` и `POSTGRES_PASSWORD` из `.env`.

Приложение обращается к базе данных не от владельца, а от отдельной учётной записи `grsn_app`, которой разрешены только чтение, добавление, изменение и удаление строк. Изменять структуру таблиц эта учётная запись не может; миграции, копирование и восстановление выполняются от владельца базы. Изменять структуру таблиц вручную не следует: структура обновляется миграциями при запуске новой версии.

База данных содержит 17 таблиц. Физическая модель базы данных представлена на рисунке 6.1.

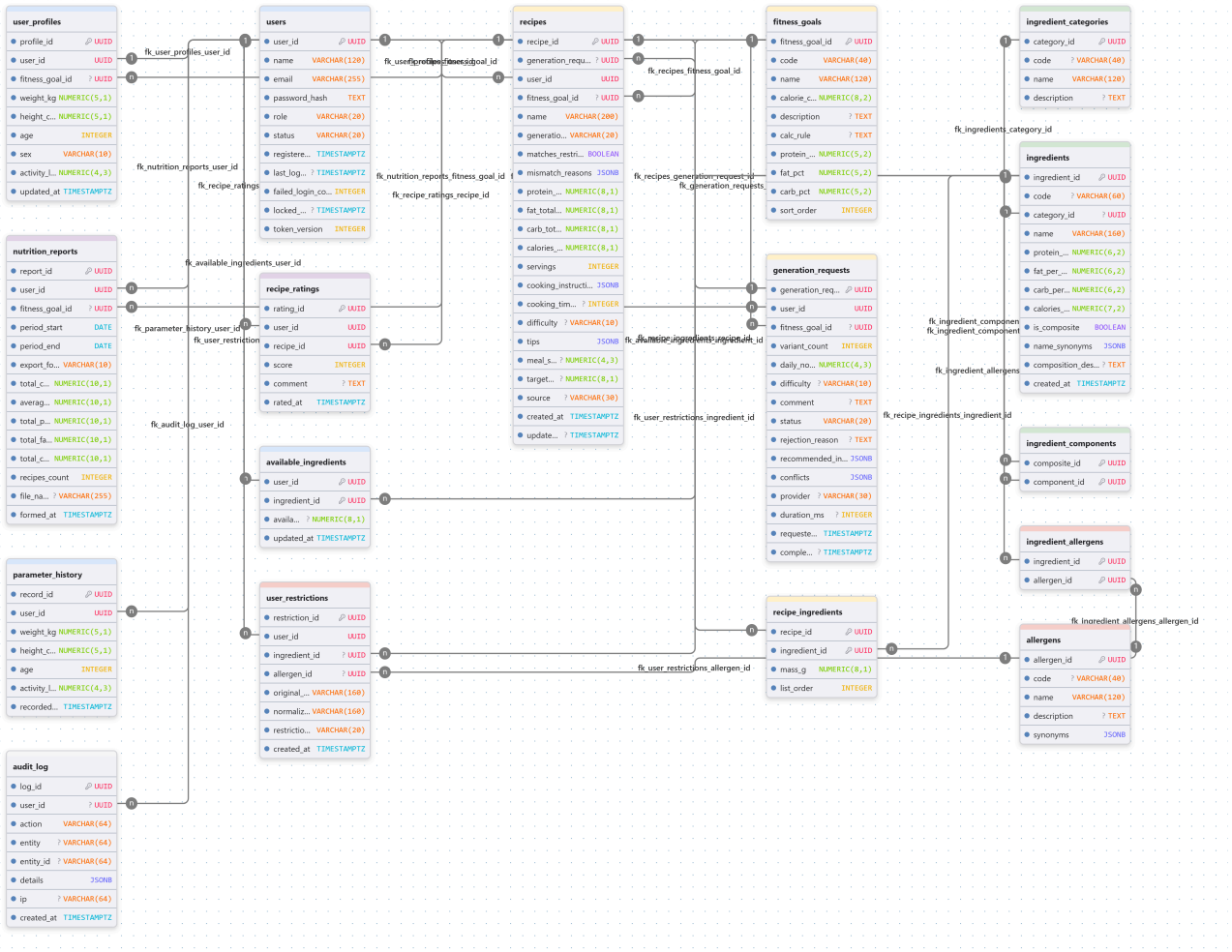


Рисунок 6.1 – Физическая модель базы данных

7 Модуль администрирования

После установки войдите в систему под учётной записью администратора. В боковом меню появится раздел «Администрирование» (рисунок 7.1). В нём доступны вкладки:

– «Пользователи» – список учётных записей; изменение роли («Сделать админом»), блокировка и удаление пользователя, просмотр его активности;
– «Ингредиенты», «Категории», «Аллергены» – добавление, изменение и удаление записей справочников;
– «Логи» – журнал событий: входы и неудачные попытки входа, блокировки, действия пользователей и администратора, резервное копирование (рисунок 7.2);
– «Резервные копии» – список копий, создание копии и восстановление (раздел 5);
– «Целостность» – проверка корректности справочников: ингредиенты без категории, сумма белков, жиров и углеводов больше 100 г на 100 г продукта, расхождение калорийности с расчётом по белкам, жирам и углеводам, ошибки в составе составных продуктов, категории без ингредиентов, аллергены, не привязанные к ингредиентам, запреты пользователей без ссылки на справочник (рисунок 7.3).

Сразу после установки рекомендуется:

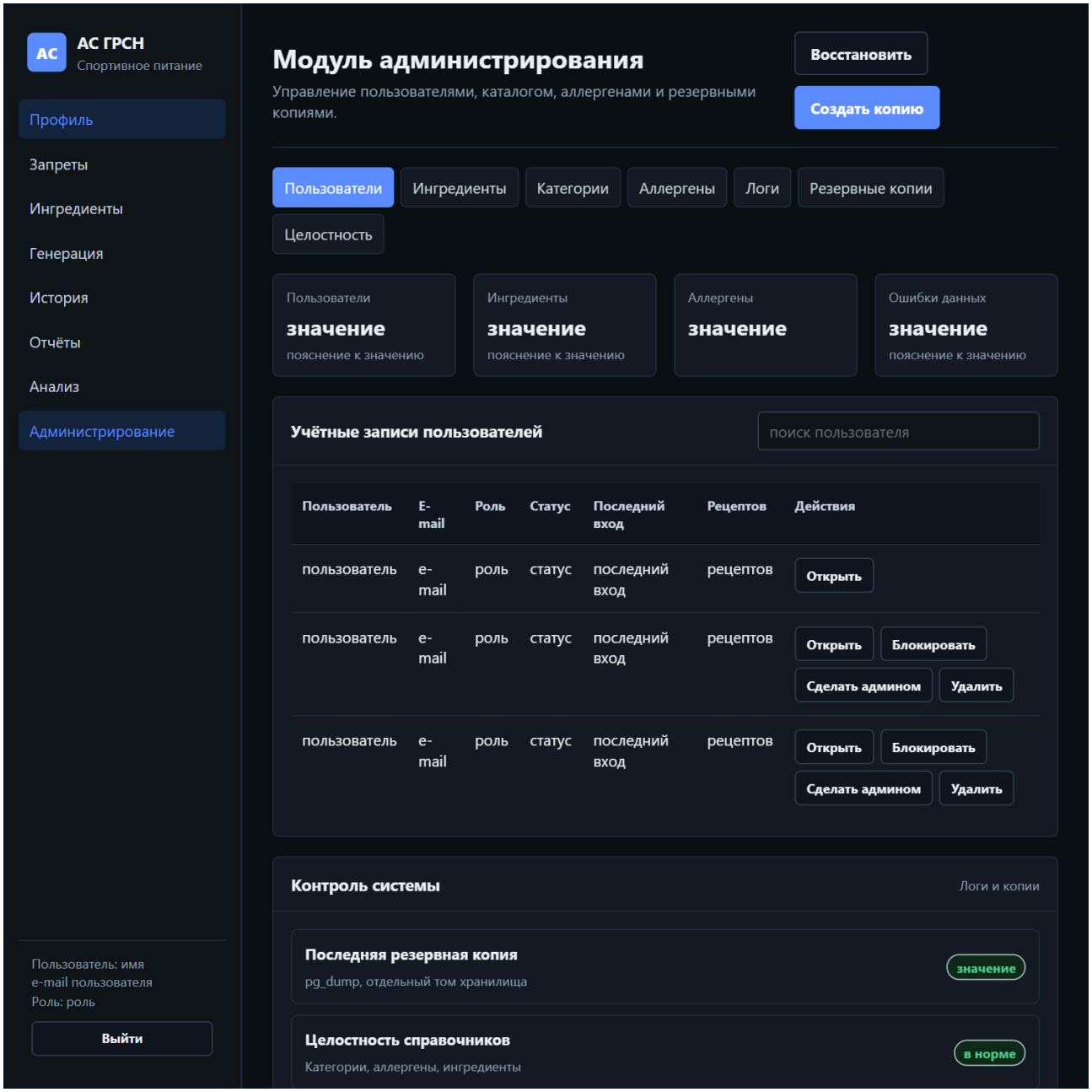


Рисунок 7.1 – Раздел «Администрирование», вкладка «Пользователи»

1) Войти под учётной записью администратора и проверить, что справочники заполнены, а вкладка «Целостность» не показывает ошибок.
2) Создать резервную копию вручную и убедиться, что файл появился в папке `BACKUP_PATH` на отдельном диске.
3) При необходимости назначить роль администратора ответственному сотруднику.

8 Обслуживание системы

Основные команды управления приведены в таблице 8.1. Команды выполняются в каталоге `grsn_app`.

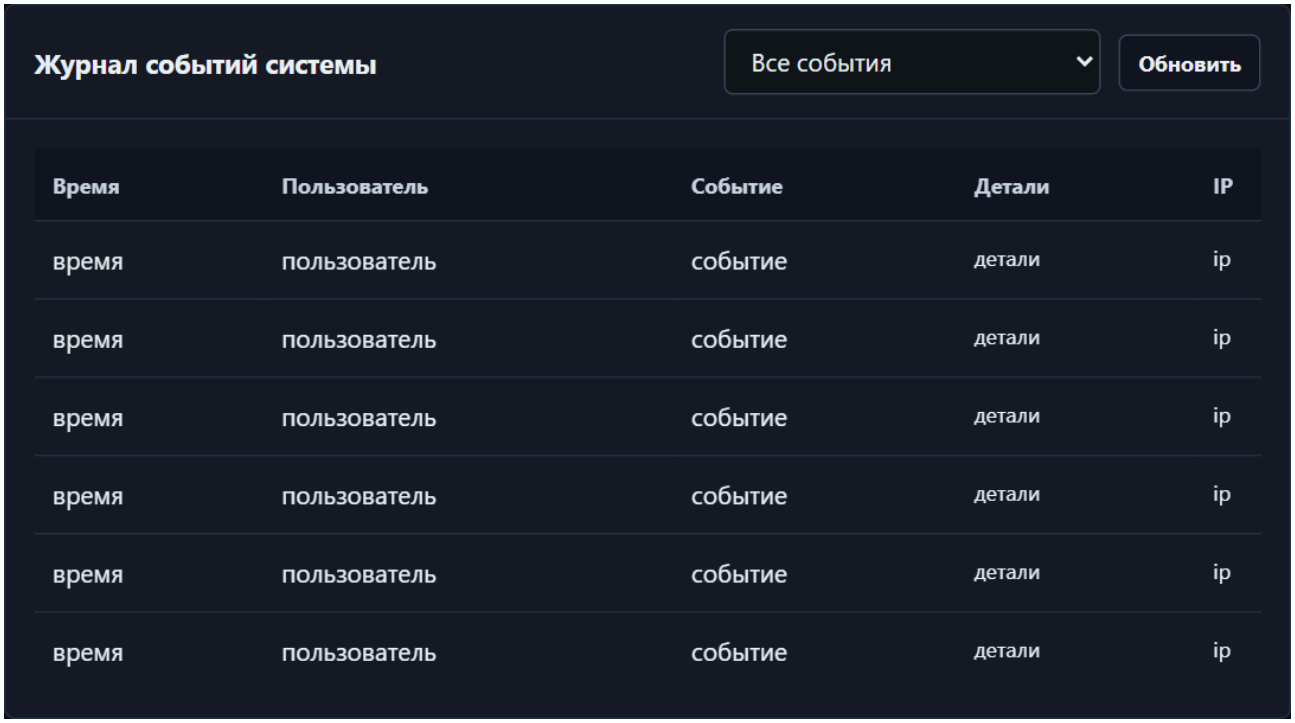


Рисунок 7.2 – Вкладка «Логи»

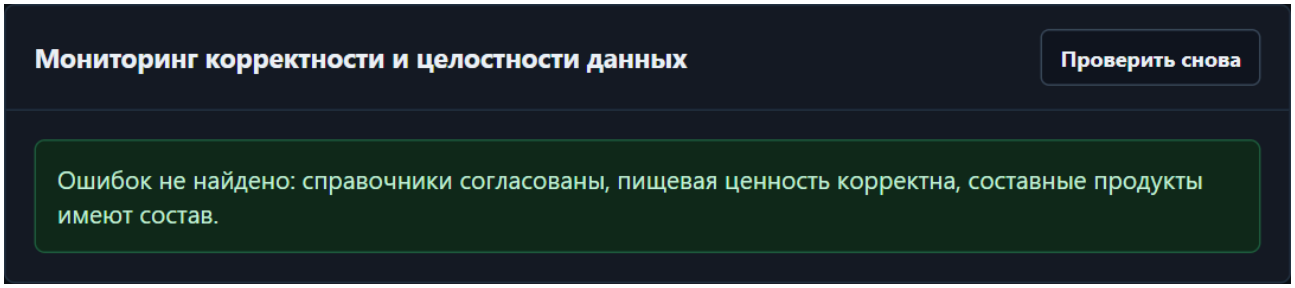


Рисунок 7.3 – Вкладка «Целостность»

Таблица 8.1 – Команды управления системой

Команда	**Назначение**
docker compose stop	Остановить систему (данные сохраняются)
docker compose up -d	Запустить систему или применить изменения .env
docker compose restart backend	Перезапустить отдельный контейнер
docker compose ps	Состояние контейнеров
docker compose logs -f backend	Журнал контейнера в реальном времени
docker stats	Потребление процессора и памяти
git pull и docker compose up -d --build	Обновление до новой версии: загрузка изменений и пересборка контейнеров; миграции базы применяются автоматически
docker compose down	Остановить и удалить контейнеры (данные в томах сохраняются)
docker compose down -v	Удалить контейнеры и **все данные**. Выполнять только после создания резервной копии

Перед обновлением системы создайте резервную копию (раздел 5.2). Контейнеры настроены на автоматический перезапуск при сбое и после перезагрузки сервера (политика `restart: unless-stopped`), если служба Docker запускается вместе с системой.

9 Типовые неисправности

Таблица 9.1 – Типовые неисправности и способы их устранения

Признак	**Причина и решение**
«Docker не найден» или «Docker не запущен»	Установите или запустите Docker (раздел 2.1) и повторите установку
docker compose up завершается ошибкой, порт занят	Укажите в .env свободные порты HTTP_PORT, HTTPS_PORT, DB_PORT либо освободите их
«Система не ответила за 5 минут»	Выполните docker compose logs backend; частые причины – нехватка памяти, неверный пароль базы данных после ручной правки .env
Сайт не открывается с других компьютеров	Откройте порты веб-интерфейса в межсетевом экране сервера
Браузер предупреждает о небезопасном соединении	Используется самоподписанный сертификат; установите сертификат, выданный для домена (раздел 3.3)
Пользователи видят предупреждение «рецепт сформирован локальным генератором»	Не задан или неверен ключ GigaChat, либо сервис недоступен: проверьте GIGACHAT_CREDENTIALS и доступ сервера к интернету, перезапустите backend
Генерация выполняется долго	Запросы к модели выполняются по очереди (тариф Freemium – один поток); при платном тарифе увеличьте LLM_MAX_CONCURRENCY
Резервная копия не создаётся, ошибка записи	Проверьте, что диск подключён и папка BACKUP_PATH доступна на запись пользователю 1001 (sudo chown 1001:1001 <папка>)
Заканчивается место на диске	Уменьшите BACKUP_RETENTION_DAYS, удалите старые образы командой docker image prune
Администратор забыл пароль, других администраторов нет	Задайте в .env новые ADMIN_EMAIL и ADMIN_PASSWORD и выполните docker compose up -d backend: будет создана новая учётная запись администратора. Затем заблокируйте или удалите старую запись на вкладке «Пользователи»